

Guide span-magnitude-viz

Guide span-magnitude-viz

Documentation claire pour comprendre, utiliser et intégrer la bibliothèque **span-magnitude-viz**.

Dépôt : github.com/michael-alteridea/span-magnitude-viz

1. À quoi sert le framework

span-magnitude-viz est une bibliothèque TypeScript / SVG pour raconter une histoire visuelle **span** × **magnitude**.

Chaque « marque » (mark) représente :

- un **intervalle** sur un axe continu (dates ISO ou nombres) — le *span* ;
- une **mesure quantitative** — la *magnitude* (budget, ACV, intensité, etc.).

Les marques apparaissent progressivement (animation en cascade), pendant que des **compteurs** (tickers) affichent le nombre d'éléments et la somme des magnitudes. Un bandeau sous le graphique propose une **synthèse par axe** (année, cohorte, groupe, champs *meta*, ...) avec moyennes et mini-arcs de quartiles.

Cas d'usage typiques : projets, contrats, subventions, carrières, abonnements, hospitalisations — tout jeu de données où chaque élément a une durée et une taille.

Inspiration de design : le storytelling temporel à marques individuelles (ex. pièces publiques de type Periscopic). La bibliothèque est **agnostique du domaine** et ne reprend ni branding, ni palette propriétaire, ni métaphores spécifiques.

2. Format de données JSON (version 1)

Le document d'entrée suit un contrat simple (**version**: 1).

Exemple minimal

```
{
  "version": 1,
  "unit": "date",
  "title": "Mon portfolio",
  "magnitudeLabel": "Budget (USD)",
  "spanLabel": "Calendrier",
  "countLabel": "Projets",
  "marks": [
    {
      "id": "p1",
      "label": "Mobile App v1",
      "span": { "start": "2020-01-15", "end": "2020-09-30" },
      "magnitude": 250000,
      "cohort": "2020",
      "group": "Product",
      "meta": { "owner": "Ada", "status": "done" }
    }
  ]
}
```

Champs importants

Champ	Requis	Rôle
version	oui	Doit être 1
unit	oui	"date" (chaînes ISO) ou "number"
marks	oui	Au moins une marque
marks[].id	oui	Identifiant unique
marks[].span.start / end	oui	Même type que unit ; end ≥ start
marks[].magnitude	oui	Nombre fini ≥ 0
revealAt	non	Instant de révélation (défaut = span.start)
cohort / group / label / meta	non	Filtre, couleur, libellé, métadonnées
defaults	non	Géométrie, palette, animation, tickers par défaut
title, *Label	non	Titre et libellés d'interface

Schéma JSON : `schema/v1.json`

Exemples : `examples/minimal-date.json`, `examples/minimal-number.json`

3. Comment l'utiliser

Installation

```
npm install span-magnitude-viz
```

Ou, en local dans le dépôt :

```
cd span-magnitude-viz
npm install
npm run build    # compile ESM + CJS + types
npm run demo     # → http://localhost:5173
```

Point d'entrée principal

```
import { createSpanMagnitudeViz, parseDocument } from "span-magnitude-viz";

const raw = await fetch("/data/projects.json").then((r) => r.json());
const chart = createSpanMagnitudeViz(
  document.getElementById("app")!,
  raw,
  {
    geometry: "arc",      // "arc" | "bar"
    width: 960,
    height: 540,
    animate: true,
    autoplay: true,
    theme: "dark",
    tickers: true,
    facetBy: "year",
    facetSummary: true,
    onHover: (mark) => console.log(mark?.label),
  }
);

chart.play();
chart.pause();
chart.reset();
chart.setGeometry("bar");
chart.setFilter({ cohort: "2021" });
chart.setFilter({ mirrorSplit: true, mirrorCohort: "2021" });
chart.setFacetBy("group");
chart.setProgress(0.5);
chart.destroy();
```

Alias : `mount( ... )` — même signature que `createSpanMagnitudeViz`.

Options importantes (VizOptions)

Option	Défaut	Rôle
geometry	"arc"	Arcs Bézier ou barres horizontales
width / height	960 / 540	Taille du SVG
animate / autoplay	true	Cascade de révélation au montage
slowFirst	2	Premières marques lentes + annotations
durationMs	adaptatif	Durée totale de l'animation
tickers	true	Compteurs (ou liste : count, magnitudeSum, spanSum)
theme	"dark"	"dark" ou "light"
cohortFilter	null	Ne montrer qu'une cohorte
mirrorSplit	false	Une cohorte au-dessus de l'axe, le reste en dessous
mirrorCohort	—	Cohorte placée au-dessus en mode miroir
facetBy	auto	Axe du bandeau (year, cohort, group, meta.*)
facetSummary / yearSummary	true	Afficher le bandeau synthèse
onHover / onSelect / onTick / onComplete	—	Callbacks d'interaction

Handle renvoyé (VizHandle)

Méthode	Rôle
<code>play()</code> / <code>pause()</code> / <code>reset()</code>	Contrôle de l'animation
<code>setGeometry(mode)</code>	Bascule arc / barre
<code>setFilter({ cohort, mirrorSplit, mirrorCohort })</code>	Filtre et miroir
<code>setFacetBy(key)</code>	Change l'axe du bandeau
<code>setProgress(t)</code>	Positionne l'horloge $t \in [0,1]$
<code>getState()</code>	État des tickers à l'instant courant
<code>update(doc, options?)</code>	Remplace les données / options
<code>destroy()</code>	Nettoie le DOM et l'animation

4. D  mo standalone / HTML offline

Plusieurs fa  ons de voir le r  sultat sans int  grer tout de suite :

1. **D  mo Vite** — `npm run demo` puis ouvrir `http://localhost:5173`
Jeux de donn  es : projets (~64 marques), contrats (~48). Contr  les : g  om  trie, cohorte, miroir, lecture/pause,   diteur JSON.
2. **Pages standalone** dans `demo/` :
 - `standalone.html` — pr  visualisation fine avec bundling Vite ;
 - `standalone-offline.html` — page HTML utilisable hors ligne (apr  s build des assets si besoin).
3. **Build de d  mo** — dossier `demo-dist/` (HTML + JS bundl  ) pour servir en statique.

Les contr  les de la d  mo incluent le s  lecteur « **D  cliner par / Break down by** » du bandeau de synth  se.

5. Synth  se par axe (facet)

Sous le graphique, un bandeau `smv-facet-summary` (activ   par d  faut) affiche **une ligne par valeur** de l'axe choisi :

- moyenne du **span** et de la **magnitude** ;

- **trois mini-arcs** (Q1, médiane, Q3) : largeur $\propto$ span, épaisseur $\propto$ magnitude — même langage visuel que le viz principal.

Sélecteur « Décliner par »

Axe	Signification
year	Années civiles chevauchées (unit: "date") ou bucket entier (unit: "number")
cohort	Champ cohort de chaque marque
group	Champ group (souvent la couleur)
meta.<clé>	Champs meta découverts automatiquement (valeurs primitives homogènes)

Défaut : year si unit $\equiv$ "date", sinon cohort ou group s'ils sont utiles.

Pour désactiver le bandeau : facetSummary: false (ou l'alias yearSummary: false).

6. Architecture des modules

```
src/
├─ index.ts          # API publique : createSpanMagnitudeViz + réexports
├─ types.ts          # Types du document, options, handle, ParseError
├─ parse.ts          # Validation Zod + normalisation
├─ layout.ts          # Échelles, chemins arc/barre, formatage
├─ animate.ts        # Timeline de révélation + requestAnimationFrame
├─ tickers.ts         # DOM des compteurs (interne au render)
├─ facetSummary.ts    # Synthèse par axe + mini-arcs + sélecteur
├─ yearSummary.ts     # Compatibilité : réexporte facetSummary
└─ render/svg.ts      # Montage SVG, interactions, orchestration
```

Flux typique :

1. **parse** — valide le JSON et produit un `NormalizedDocument` ;
2. **layout** — calcule positions, couleurs, chemins SVG ;
3. **animate** — construit le calendrier de révélation et pilote les frames ;
4. **render/svg** — dessine, attache tickers + bandeau facet, gère hover/filtre ;
5. **facetSummary** — agrège moyennes / quartiles et peint le bandeau.

7. Catalogue des fonctions exportées

Fonctions et types publics depuis `src/index.ts`, regroupés par module.
Paramètres principaux et valeur de retour indiqués brièvement.

Entrée principale (`index.ts`)

`createSpanMagnitudeViz(container, document, options?)`

Monte la visualisation dans un élément DOM. Accepte un document brut ou déjà normalisé.
Remplace le contenu du conteneur. **Retour** : `VizHandle`.

`mount`

Alias de `createSpanMagnitudeViz` (même signature). Export aussi comme `default`.

Types & erreurs (`types.ts`)

Types utiles : `SpanUnit`, `GeometryMode`, `TickerKind`, `SpanEndpoints`, `SpanMark`,
`SpanMagnitudeDefaults`, `SpanMagnitudeDocument`, `NormalizedMark`, `NormalizedDocument`,
`LayoutMark`, `VizOptions`, `TickerState`, `VizHandle`.

`ParseError`

Erreur de validation. Propriété `issues` : `string[]` (messages multilignes). Levée par `parseDocument`.

Parsing (`parse.ts`)

`parseDocument(input, options?)`

Valide le contrat v1 (Zod), convertit dates → millisecondes UTC, calcule domaines, cohorts, groups.
Option `{ strict: true }` refuse les clés inconnues. **Retour** : `NormalizedDocument`. **Lève** :
`ParseError`.

`tryParseDocument(input, options?)`

Même logique sans exception. **Retour** : `{ ok: true, data }` ou `{ ok: false, error: ParseError }`.

Layout (`layout.ts`)

`computeLayout(doc, options, visibleMarks?)`

Calcule échelles X, positions, épaisseurs, couleurs, chemins d'arc et rectangles de barre. Respecte
`mirrorSplit` / `mirrorCohort`. **Retour** : `LayoutResult` (marques layoutées + métadonnées de canvas).

arcPath(x0, x1, y, bulge, side)

Chaîne SVG d'une courbe de Bézier quadratique entre deux points sur une baseline. **side** = 1 (haut) ou -1 (bas). **Retour** : string (d path).

formatAxisValue(v, unit)

Formate une valeur d'axe (YYYY-MM pour les dates, nombre sinon). **Retour** : string.

formatMagnitude(v)

Formate une magnitude compacte (1.2k, 3.4M, ...). **Retour** : string.

formatSpanRange(start, end, unit)

Libellé d'intervalle (2020-01-15 → 2020-09-30 ou 10 → 42). **Retour** : string.

Animation (animate.ts)

buildRevealSchedule(marks, { durationMs?, slowFirst? })

Construit la timeline de révélation : les **slowFirst** premières marques occupent ~28 % du temps, le reste cascade jusqu'à 0,95. **Retour** : RevealSchedule.

markProgress(schedule, id, t)

Progression de dessin d'une marque à l'horloge $t \in [0,1]$ (smoothstep entre t_0 et t_1). **Retour** : number $\in [0,1]$.

tickerAt(schedule, t)

Agrège compteur, somme des magnitudes et somme des spans selon la progression. **Retour** : TickerState.

createAnimation(schedule, onFrame, onComplete?)

Contrôleur requestAnimationFrame : play, pause, reset, setProgress, getProgress, destroy. **Retour** : AnimationController.

prefersReducedMotion()

Lit prefers-reduced-motion : reduce du navigateur. **Retour** : boolean.

Rendu SVG (render/svg.ts)

mountSvg(container, docInput, options?)

Orchestrateur principal : styles, layout, SVG D3, tickers, bandeau facet, animation, tooltips, filtres. Utilisé en interne par createSpanMagnitudeViz. **Retour** : VizHandle.

Synthèse par axe (facetSummary.ts)

quantileSorted(sorted, p)

Quantile linéaire (style R-7) sur un tableau **déjà trié**. $p \in [0,1]$. **Retour** : number.

yearsForMark(mark, unit)

Années auxquelles une marque contribue (chevauchement calendaire + cohorte 4 chiffres ; ou bucket entier en mode nombre). **Retour** : number[].

keysForMark(mark, facetBy, unit)

Clés de bucket pour un axe (year, cohort, group, meta.*). **Retour** : string[].

discoverFacetAxes(marks, unit)

Liste les axes disponibles pour le sélecteur (built-ins + meta.* homogènes). **Retour** : FacetAxis[].

defaultFacetBy(doc, marks?)

Choisit l'axe par défaut (year si dates, sinon cohort/group utiles). **Retour** : string.

resolveFacetBy(doc, marks, requested?)

Valide une demande d'axe ou retombe sur le défaut. **Retour** : string.

computeFacetSummaries(marks, facetBy, unit)

Calcule une ligne par bucket : compte, moyennes span/magnitude, quartiles Q1/Q2/Q3. **Retour** : FacetSummaryRow[].

computeYearSummaries(marks, unit)

Compatibilité : équivalent à computeFacetSummaries(... , "year", unit) avec un champ year numérique. **Retour** : YearSummaryRow[].

formatAvgSpan(avgSpan, unit)

Formate une moyenne de span (12d, 1.5y, ou magnitude numérique). **Retour** : string.

createFacetSummaryDom(container, doc, marks, theme, options?)

Crée le bandeau HTML (titre, sélecteur « Décliner par », lignes, mini-arcs). Options : facetBy, onFacetChange, showPicker. **Retour** : { root }.

Types exportés : FacetAxis, FacetSummaryRow, FacetQuartileArc, FacetSummaryElements, FacetSummaryDomOptions.

Compatibilité année (yearSummary.ts)

Module mince qui **réexporte** les symboles de facetSummary (dont createYearSummaryDom, YEAR_SUMMARY_CSS). À utiliser seulement pour du code ancien ; préférer facetSummary / les exports de index.ts.

`createYearSummaryDom( ... )` (via `facetSummary`, non réexporté dans `index.ts`)

Alias déprécié : bandeau forcé sur `year` sans sélecteur.

Internes utiles (non exportés par `index.ts`)

Ces helpers vivent dans `tickers.ts` et sont branchés par le `render` :

- `createTickerDom` — crée le DOM des compteurs ;
- `updateTickers` — met à jour les valeurs à chaque frame ;
- `TICKER_CSS` — styles injectés avec le thème.

Vous n'avez en général **pas besoin** de les appeler : `createSpanMagnitudeViz` s'en charge.

8. Géométries

- `arc` (défaut) — courbes de Bézier sur une baseline ; épaisseur $\propto$ magnitude ; léger jitter pour étaler les arcs.
 - `bar` — barres horizontales de `x0` à `x1`, hauteur $\propto$ magnitude.
 - `lane` — réservé ; actuellement traité comme `bar`.
-

9. Intégration rapide — checklist

1. Préparer un JSON `version: 1` avec `unit` et `marks`.
 2. Valider avec `tryParseDocument` si vous voulez gérer les erreurs proprement.
 3. Appeler `createSpanMagnitudeViz(el, data, options)`.
 4. Brancher `play` / `pause` / `filtres` selon votre UI.
 5. Activer ou configurer le bandeau via `facetBy` / `facetSummary`.
 6. Pour une page offline, s'appuyer sur `demo/standalone-offline.html` ou un build Vite.
-

10. Licence & liens

- Licence : **MIT**
- GitHub : <https://github.com/michael-alteridea/span-magnitude-viz>
- Design détaillé : `DESIGN.md` à la racine du dépôt
- README : `README.md`

Bonne intégration — et n’hésitez pas à partir des exemples ~~exemples~~/ pour un premier prototype en quelques minutes.